

运行与维护

Violet

2026-04-25

Chapter 0

目 录

I	Linux 系统运维基础	
1.	Linux 系统管理	6
1.1.	系统监控	6
1.2.	进程管理	8
1.3.	文件系统管理	11
1.4.	用户与权限	13
1.5.	日志管理	15
1.6.	网络配置	17
2.	Shell 脚本自动化	20
2.1.	Shell 编程进阶	20
2.2.	文本处理	23
2.3.	定时任务	26
2.4.	系统备份	28
2.5.	自动化部署脚本	31
3.	服务管理与高可用	37
II	容器化技术	
4.	Docker 基础	39
4.1.	容器概念	39
4.2.	Docker 安装与配置	41
4.3.	镜像管理	43
4.4.	容器操作	46
4.5.	数据持久化	49
4.6.	网络模式	51
5.	Docker 高级特性	53
6.	容器运行时	54
III	云原生基础	
7.	云原生理念与架构	56
8.	服务网格 (Service Mesh)	57

9. 云原生存储 58

IV Kubernetes 核心

10. K8s 架构与核心概念 60

11. 工作负载管理 61

12. 服务发现与网络 62

13. 存储与配置管理 63

14. 安全与权限控制 64

V Kubernetes 进阶

15. Helm 包管理 66

16. Operator 模式 67

17. 监控与日志 68

18. CI/CD 与 GitOps 69

VI 云平台与基础设施

19. 公有云 K8s 服务 71

20. 基础设施即代码 (IaC) 72

21. 边缘计算与 K8s 73

VII 生产实践与故障排查

22. 集群规划与部署 75

23. 性能优化 76

24. 故障排查方法论 77

25. 成本优化与管理 78



Linux 系统运维基础

1. Linux 系统管理	6
2. Shell 脚本自动化	20
3. 服务管理与高可用	37

Chapter 1

Linux 系统管理

Linux 系统管理是运维工作的基础，包括系统监控、进程管理、文件系统、用户权限、日志和网络配置等核心技能。

1 系统监控

系统监控帮助了解服务器运行状态，及时发现性能瓶颈和异常。

1.1 top/htop：实时进程监控

```
1  # top: 系统自带
2
3  top
4
5  # 常用快捷键
6  # P: 按 CPU 使用率排序
7  # M: 按内存使用率排序
8  # 1: 显示所有 CPU 核心
9  # q: 退出
10
11 # htop: 增强版 (需安装)
12
13 htop
14
15 # 优势:
16 # - 彩色显示
17 # - 支持鼠标操作
18 # - 树状视图
19 # - 更直观的界面
```

top 输出解读:

字段	含义	说明
PID	进程 ID	唯一标识
USER	用户	进程所有者
PR	优先级	数值越小优先级越高
NI	Nice 值	-20 到 19，影响优先级
VIRT	虚拟内存	进程使用的虚拟内存

RES	resident 内存	实际使用的物理内存
SHR	共享内存	可与其他进程共享的内存
S	状态	R 运行/S 睡眠/Z 僵尸
%CPU	CPU 使用率	百分比
%MEM	内存使用率	百分比
TIME+	CPU 时间	进程占用的 CPU 时间
COMMAND	命令	进程名称

TIP

生产环境推荐使用 `htop`，界面更友好，功能更强大。

1.2 vmstat：虚拟内存统计

```
1 # 每2秒刷新一次，共5次
2
3 vmstat 2 5
4
5 # 输出示例：
6 # procs -----memory----- --swap-- -----io----- -system-- -----cpu-----
7 #  r  b   swpd   free   buff  cache   si   so    bi    bo    in   cs us sy id wa st
8 #  1  0       0 123456  12345 678901   0   0    10    20  100  200  5  2 93  0  0
```

关键指标：

列	含义	正常范围
r	运行队列长度	< CPU 核心数
b	阻塞进程数	0
swpd	交换分区使用	越低越好
free	空闲内存	越高越好
si/so	swap in/out	应为 0
bi/bo	块设备 IO	根据负载
us	用户 CPU	< 70%
sy	系统 CPU	< 30%
id	空闲 CPU	越高越好
wa	IO 等待	< 10%

CAUTION

如果 `si/so` 不为 0，说明内存不足，正在使用 swap，应增加内存或优化应用。

1.3 iostat：磁盘 IO 监控

```
1 # 安装 sysstat 包
2
3 sudo apt install sysstat # Debian/Ubuntu
4 sudo yum install sysstat # CentOS/RHEL
5
6 # 每2秒刷新一次
7
8 iostat -x 2
9
10 # 输出示例:
11 # Device rrqm/s wrqm/s r/s w/s rkB/s wkB/s avgrq-sz avgqu-sz await svctm %util
12 # sda      0.00  1.00  0.50  2.00  10    20    20.00    0.01    2.00  1.00  0.25
```

关键指标:

- `%util`: 磁盘利用率, 接近 100% 表示磁盘饱和
- `await`: IO 请求平均等待时间, 过高表示磁盘瓶颈
- `svctm`: IO 请求平均服务时间
- `avgqu-sz`: 平均队列长度

TIP

`%util` 高但 `await` 低, 说明磁盘繁忙但性能好; 两者都高说明磁盘成为瓶颈。

1.4 sar: 系统活动报告

```
1 # 查看历史数据
2
3 sar -u # CPU 使用率
4 sar -r # 内存使用
5 sar -b # IO 传输
6 sar -n DEV # 网络统计
7 sar -q # 队列长度和负载
8
9 # 查看特定时间段
10
11 sar -u -f /var/log/sa/sa15 # 查看15日的数据
12
13 # 实时监控
14
15 sar -u 1 5 # 每1秒采样, 共5次
```

sar 优势:

- 可以查看历史数据 (默认保存 7 天)
- 数据存储在于 `/var/log/sa/`
- 支持多种指标 (CPU、内存、IO、网络)

2 进程管理

2.1 ps: 进程快照

```
1 # 查看所有进程
2
3 ps aux
4
5 # 输出字段:
6 # USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
7
8 # 查找特定进程
9
10 ps aux | grep nginx
11 ps -ef | grep java
12
13 # 按条件筛选
14
15 ps aux --sort=-%cpu | head -10 # CPU 使用率前10
16 ps aux --sort=-%mem | head -10 # 内存使用率前10
17
18 # 查看进程树
19
20 ps auxf # 树状显示
21 pstree # 更清晰的树状图
```

进程状态:

状态	含义
R	运行中
S	可中断睡眠
D	不可中断睡眠
Z	僵尸进程
T	停止/追踪

2.2 kill: 终止进程

```
1 # 发送信号
2
3 kill 1234 # 默认 SIGTERM (优雅终止)
4 kill -9 1234 # SIGKILL (强制终止)
5 kill -15 1234 # SIGTERM (显式指定)
6 kill -1 1234 # SIGHUP (重新加载配置)
7
8 # 批量终止
9
10 killall nginx # 按名称终止
11 pkill -f "java.*app" # 按模式匹配
12
13 # 查看信号列表
14
15 kill -l
```

常用信号：

- `SIGTERM` (15)：请求终止，允许清理资源
- `SIGKILL` (9)：强制终止，无法捕获
- `SIGHUP` (1)：重新加载配置
- `SIGINT` (2)：中断 (Ctrl+C)
- `SIGSTOP` (19)：暂停
- `SIGCONT` (18)：继续

CAUTION

优先使用 `SIGTERM`，给进程清理资源的机会。`SIGKILL` 是最后手段。

2.3 nice/renice：调整优先级

```
1 # nice：启动时设置优先级
2
3 nice -n 10 ./slow_script.sh # 较低优先级
4 nice -n -5 ./important_task # 较高优先级（需要 root）
5
6 # renice：修改运行中进程的优先级
7
8 renice -n 10 -p 1234 # 降低优先级
9 renice -n -5 -p 1234 # 提高优先级（需要 root）
10
11 # 优先级范围：-20（最高）到 19（最低）
```

NOTE

Nice 值越低，优先级越高。普通用户只能降低优先级（增大 nice 值）。

2.4 systemd：现代 init 系统

```
1 # 服务管理
2
3 systemctl start nginx # 启动服务
4 systemctl stop nginx # 停止服务
5 systemctl restart nginx # 重启服务
6 systemctl reload nginx # 重载配置
7 systemctl status nginx # 查看状态
8
9 # 开机自启
10
11 systemctl enable nginx # 启用开机自启
12 systemctl disable nginx # 禁用开机自启
13
14 # 查看日志
15
16 journalctl -u nginx # 查看服务日志
17 journalctl -u nginx -f # 实时跟踪
18 journalctl -u nginx --since "1 hour ago"
```

```
19
20 # 查看所有服务
21
22 systemctl list-units --type=service
23 systemctl list-unit-files # 查看所有单元文件
```

systemd 优势:

- 并行启动，加快开机速度
- 依赖管理，自动处理服务顺序
- 统一的日志管理 (journalctl)
- 强大的资源配置能力

3 文件系统管理

3.1 df: 磁盘空间使用情况

```
1 # 查看所有挂载点
2
3 df -h # 人类可读格式
4
5 # 输出示例:
6 # Filesystem      Size  Used Avail Use% Mounted on
7 # /dev/sda1        50G   20G   30G   40% /
8 # tmpfs            7.8G    0   7.8G    0% /dev/shm
9
10 # 查看特定类型
11
12 df -hT           # 显示文件系统类型
13 df -h /home      # 查看特定目录
```

TIP

定期监控磁盘使用率，建议在达到 80% 时发出警告，90% 时紧急处理。

3.2 du: 目录大小统计

```
1 # 查看目录大小
2
3 du -sh /var/log # 单个目录总大小
4 du -sh /var/*   # 子目录大小
5
6 # 查找大文件
7
8 du -ah /var | sort -rh | head -20 # 最大的20个文件/目录
9
10 # 深度控制
11
12 du -h --max-depth=1 /home # 只显示第一层
```

3.3 fdisk: 分区管理

```
1 # 查看分区表
2
3 sudo fdisk -l
4
5 # 交互式分区
6
7 sudo fdisk /dev/sdb
8
9 # 常用命令:
10 # p: 打印分区表
11 # n: 新建分区
12 # d: 删除分区
13 # t: 修改分区类型
14 # w: 写入并退出
15 # q: 退出不保存
```

CAUTION

分区操作有风险，务必先备份重要数据。生产环境建议使用 LVM。

3.4 LVM: 逻辑卷管理

LVM (Logical Volume Manager) 提供灵活的存储管理。

```
1 # 创建物理卷
2
3 pvcreate /dev/sdb1
4
5 # 创建卷组
6
7 vgcreate vg_data /dev/sdb1
8
9 # 创建逻辑卷
10
11 lvcreate -L 100G -n lv_home vg_data
12
13 # 格式化并挂载
14
15 mkfs.ext4 /dev/vg_data/lv_home
16 mount /dev/vg_data/lv_home /home
17
18 # 扩展逻辑卷 (在线)
19
20 lvextend -L +50G /dev/vg_data/lv_home
21 resize2fs /dev/vg_data/lv_home # ext4
22 xfs_growfs /home # xfs
23
24 # 查看信息
25
26 pvdisplay
```

```
27 vgdisplay
28 lvdisplay
```

LVM 优势:

- 在线扩展/缩小卷
- 快照功能（备份）
- 跨多个磁盘聚合存储
- 灵活的存储池管理

4 用户与权限

4.1 用户管理

```
1 # 创建用户
2
3 sudo useradd -m -s /bin/bash username
4 sudo passwd username
5
6 # 删除用户
7
8 sudo userdel -r username # -r 同时删除家目录
9
10 # 修改用户
11
12 sudo usermod -aG sudo username # 添加到 sudo 组
13 sudo usermod -s /sbin/nologin username # 禁止登录
14
15 # 查看用户
16
17 id username
18 whoami
19 cat /etc/passwd
```

TIP

`-m` 创建家目录, `-s` 指定 shell, `-aG` 追加到组（不覆盖原有组）。

4.2 文件权限

```
1 # 查看权限
2
3 ls -l file.txt
4 # -rw-r--r-- 1 user group 1024 Jan 1 12:00 file.txt
5
6 # 权限分解:
7 # -   rw-   r--   r--
8 # |   |     |     |
9 # 类型 所有者 所属组 其他用户
10
```

```

11 # 修改权限 (符号模式)
12
13 chmod u+x script.sh      # 所有者添加执行权限
14 chmod g-w file.txt       # 组移除写入权限
15 chmod o=r file.txt       # 其他用户只读
16 chmod a+x script.sh      # 所有用户添加执行权限
17
18 # 修改权限 (数字模式)
19
20 chmod 755 script.sh      # rwxr-xr-x
21 chmod 644 file.txt       # rw-r--r--
22 chmod 600 private.key    # rw-----

```

数字权限对照：

权限	数字	含义
r	4	读
w	2	写
x	1	执行
rwx	7	4+2+1
rw-	6	4+2
r-x	5	4+1

4.3 chown: 修改所有者

```

1 # 修改所有者
2
3 sudo chown newuser file.txt
4 sudo chown newuser:newgroup file.txt # 同时修改组和所有者
5
6 # 递归修改
7
8 sudo chown -R user:group /directory/

```

 Bash

4.4 sudo: 提权执行

```

1 # 使用 sudo
2
3 sudo command
4
5 # 编辑 sudoers 文件 (必须用 visudo)
6
7 sudo visudo
8
9 # 添加规则
10
11 username ALL=(ALL:ALL) ALL

```

 Bash

```
12 %admin ALL=(ALL) NOPASSWD: ALL # admin 组无需密码
13
14 # 查看 sudo 权限
15
16 sudo -l
```

CAUTION

始终使用 `visudo` 编辑 `sudoers` 文件，它会检查语法错误，避免锁定系统。

5 日志管理

5.1 journalctl: systemd 日志

```
1 # 查看所有日志
2
3 journalctl
4
5 # 查看特定服务
6
7 journalctl -u nginx
8 journalctl -u sshd
9
10 # 时间过滤
11
12 journalctl --since "2024-01-01"
13 journalctl --since "1 hour ago"
14 journalctl --until "10 min ago"
15
16 # 优先级过滤
17
18 journalctl -p err # 错误及以上
19 journalctl -p warning # 警告及以上
20
21 # 实时跟踪
22
23 journalctl -f
24 journalctl -u nginx -f
25
26 # 查看内核日志
27
28 journalctl -k
```

TIP

`journalctl` 是 `systemd` 系统的统一日志接口，取代了传统的 `syslog`。

5.2 logrotate: 日志轮转

```
1 # 配置文件
```

```

2
3  /etc/logrotate.conf          # 主配置
4  /etc/logrotate.d/           # 子配置目录
5
6  # 示例配置 (/etc/logrotate.d/nginx)
7
8  /var/log/nginx/*.log {
9      daily                    # 每天轮转
10     missingok                 # 日志不存在不报错
11     rotate 14                 # 保留14个备份
12     compress                  # 压缩旧日志
13     delaycompress              # 延迟一天压缩
14     notifempty                 # 空文件不轮转
15     create 0640 www-data adm
16     sharedscripts
17     postrotate
18         [ -f /var/run/nginx.pid ] && kill -USR1 `cat /var/run/nginx.pid`
19     endscript
20 }
21
22 # 手动测试
23
24 logrotate -d /etc/logrotate.d/nginx # 调试模式
25 logrotate -f /etc/logrotate.d/nginx # 强制执行

```

常用选项：

- `daily/weekly/monthly`：轮转频率
- `rotate N`：保留 N 个备份
- `size N`：达到指定大小后轮转
- `compress`：压缩旧日志
- `copytruncate`：复制后清空（适合不支持 reopen 的应用）

5.3 rsyslog：传统日志系统

```

1  # 配置文件
2
3  /etc/rsyslog.conf
4  /etc/rsyslog.d/
5
6  # 基本规则
7
8  # 格式: facility.priority    action
9  *.info;mail.none;authpriv.none;cron.none    /var/log/messages
10 authpriv.*                                     /var/log/secure
11 mail.*                                         /var/log/maillog
12 cron.*                                         /var/log/cron
13
14 # 远程日志
15
16 *.* @remote-host:514    # TCP
17 *.* @remote-host:514    # UDP

```

```
18
19 # 重启服务
20
21 sudo systemctl restart rsyslog
```

NOTE

现代 Linux 发行版通常同时运行 rsyslog 和 journald, journald 作为主要日志系统。

6 网络配置

6.1 ip: 网络接口管理

```
1 # 查看网络接口  Bash
2
3 ip addr show
4 ip -brief addr show      # 简洁输出
5
6 # 查看路由表
7
8 ip route show
9 ip route get 8.8.8.8     # 查询特定IP的路由
10
11 # 查看邻居表 (ARP)
12
13 ip neigh show
14
15 # 启用/禁用接口
16
17 sudo ip link set eth0 up
18 sudo ip link set eth0 down
19
20 # 添加IP地址
21
22 sudo ip addr add 192.168.1.100/24 dev eth0
23 sudo ip addr del 192.168.1.100/24 dev eth0
```

TIP

`ip` 命令取代了旧的 `ifconfig`、`route`、`arp` 等命令, 功能更强大。

6.2 ss: Socket 统计

```
1 # 查看所有连接  Bash
2
3 ss -tulpn
4
5 # 输出示例:
6 # Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
7 # tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:* users:(...)
```

```

8 # tcp ESTAB 0 0 192.168.1.10:22 192.168.1.5:54321
9
10 # 过滤
11
12 ss -t state established # 已建立的TCP连接
13 ss -t state listening # 监听端口
14 ss -u # UDP连接
15 ss -x # Unix socket
16
17 # 统计
18
19 ss -s # 摘要统计

```

ss vs netstat:

- `ss` 更快，直接从内核获取信息
- `netstat` 已废弃，但仍在广泛使用
- 推荐使用 `ss`

6.3 iptables: 防火墙

```

1 # 查看规则
2
3 sudo iptables -L -n -v
4 sudo iptables -L INPUT -n -v
5
6 # 基本规则
7
8 sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT # 允许SSH
9 sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT # 允许HTTP
10 sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT # 允许HTTPS
11 sudo iptables -A INPUT -j DROP # 拒绝其他
12
13 # 删除规则
14
15 sudo iptables -D INPUT 3 # 删除第3条规则
16
17 # 保存规则 (Debian/Ubuntu)
18
19 sudo iptables-save > /etc/iptables/rules.v4
20
21 # 保存规则 (CentOS/RHEL)
22
23 sudo service iptables save

```

CAUTION

iptables 规则复杂且容易出错，生产环境建议使用 `firewalld` 或 `ufw` 简化管理。

6.4 NetworkManager: 网络管理

```

1 # 使用 nmcli

```

```
2
3 nmcli device status      # 查看设备状态
4 nmcli connection show   # 查看连接
5 nmcli device wifi list   # 扫描WiFi
6
7 # 配置静态IP
8
9 sudo nmcli con mod "eth0" ipv4.addresses 192.168.1.100/24
10 sudo nmcli con mod "eth0" ipv4.gateway 192.168.1.1
11 sudo nmcli con mod "eth0" ipv4.dns "8.8.8.8 8.8.4.4"
12 sudo nmcli con mod "eth0" ipv4.method manual
13 sudo nmcli con up "eth0"
14
15 # 使用 nmtui (文本界面)
16
17 sudo nmtui
```

本章完

Chapter 2

Shell 脚本自动化

Shell 脚本是 Linux 运维自动化的核心工具，通过编写脚本可以批量执行任务、定时调度、系统备份等。

1 Shell 编程进阶

1.1 函数定义与调用

```
1  #!/bin/bash
2
3  # 基本函数
4
5  greet() {
6      echo "Hello, $1!"
7  }
8
9  # 调用函数
10
11 greet "Alice"
12 greet "Bob"
13
14 # 带返回值
15
16 add() {
17     local result=$(( $1 + $2 ))
18     echo $result
19 }
20
21 sum=$(add 5 3)
22 echo "Sum: $sum" # Sum: 8
23
24 # 检查函数是否存在
25
26 if declare -f greet > /dev/null; then
27     echo "Function greet exists"
28 fi
```

TIP

使用 `local` 关键字声明局部变量，避免污染全局命名空间。

1.2 数组操作

```
1  #!/bin/bash
2
3  # 定义数组
4
5  fruits=("apple" "banana" "cherry")
6  numbers=(1 2 3 4 5)
7
8  # 访问元素
9
10 echo ${fruits[0]}      # apple
11 echo ${fruits[@]}      # 所有元素
12 echo ${#fruits[@]}     # 数组长度 (3)
13
14 # 修改数组
15
16 fruits[1]="orange"     # 修改元素
17 fruits+=("grape")      # 追加元素
18
19 # 遍历数组
20
21 for fruit in "${fruits[@]"; do
22     echo "Fruit: $fruit"
23 done
24
25 # 带索引遍历
26
27 for i in "${!fruits[@]"; do
28     echo "$i: ${fruits[$i]}"
29 done
30
31 # 切片
32
33 echo ${fruits[@]:1:2}   # 从索引1开始，取2个元素
34
35 # 关联数组 (Bash 4+)
36
37 declare -A colors
38 colors["red"]="FF0000"
39 colors["green"]="00FF00"
40 colors["blue"]="0000FF"
41
42 for key in "${!colors[@]"; do
43     echo "$key: ${colors[$key]}"
44 done
```

1.3 正则表达式

```
1  #!/bin/bash
2
3  # 基本匹配
4
```

```
5  if [[ "hello world" =~ ^hello ]]; then
6      echo "Starts with hello"
7  fi
8
9  # 提取匹配内容
10
11 email="user@example.com"
12 if [[ $email =~ ([a-zA-Z0-9._%+-]+)@([a-zA-Z0-9.-]+\.[a-zA-Z]{2,}) ]]; then
13     echo "Username: ${BASH_REMATCH[1]}"
14     echo "Domain: ${BASH_REMATCH[2]}"
15     echo "TLD: ${BASH_REMATCH[3]}"
16 fi
17
18 # 常用正则模式
19
20 # 验证IP地址
21 ip_pattern="^[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}$"
22 if [[ "192.168.1.1" =~ $ip_pattern ]]; then
23     echo "Valid IP"
24 fi
25
26 # 验证日期格式
27 date_pattern="^[0-9]{4}-[0-9]{2}-[0-9]{2}$"
28 if [[ "2024-01-15" =~ $date_pattern ]]; then
29     echo "Valid date format"
30 fi
31
32 # 验证手机号
33 phone_pattern="^[1-9][3-9][0-9]{9}$"
34 if [[ "13812345678" =~ $phone_pattern ]]; then
35     echo "Valid phone number"
36 fi
```

NOTE

Bash 的正则使用 `=~` 运算符，匹配结果存储在 `BASH_REMATCH` 数组中。

1.4 字符串处理

```
1  #!/bin/bash
2
3  str="Hello World"
4
5  # 长度
6
7  echo ${#str}          # 11
8
9  # 截取
10
11 echo ${str:0:5}        # Hello
12 echo ${str:6}          # World
13 echo ${str: -5}        # World (注意空格)
```

 Bash

```
14
15 # 替换
16
17 echo ${str/World/Unix} # Hello Unix
18 echo ${str//l/L}       # HeLlO WorLd (全部替换)
19
20 # 删除
21
22 echo ${str#Hello }     # World (删除前缀)
23 echo ${str% World}     # Hello (删除后缀)
24
25 # 大小写转换
26
27 echo ${str,,}          # hello world (转小写)
28 echo ${str^^}          # HELLO WORLD (转大写)
29 echo ${str^}           # Hello world (首字母大写)
30
31 # 默认值
32
33 var=""
34 echo ${var:-default}   # default (如果var为空, 使用default)
35 echo ${var:=default}   # 同时赋值
36 echo ${var:+exists}    # 如果var非空, 返回exists
```

2 文本处理

2.1 awk 高级用法

```
1  #!/bin/bash
2
3  # 内置变量
4
5  awk '{print NR, NF, $0}' file.txt # NR:行号, NF:字段数
6
7  # 条件过滤
8
9  awk '$3 > 100 {print $1, $3}' data.txt
10
11 # 数学运算
12
13 awk '{sum += $1} END {print "Sum:", sum, "Avg:", sum/NR}' numbers.txt
14
15 # 格式化输出
16
17 awk '{printf "%-10s %5d %8.2f\n", $1, $2, $3}' data.txt
18
19 # 多分隔符
20
21 awk -F'[: ]' '{print $1, $3}' file.txt # 以:或空格分隔
```

```
22
23 # 关联数组
24
25 awk '{count[$1]++} END {for (word in count) print word, count[word]}' words.txt
26
27 # 外部变量
28
29 threshold=100
30 awk -v thresh=$threshold '$1 > thresh {print $0}' data.txt
31
32 # 实战：分析日志
33
34 # 统计HTTP状态码
35 awk '{print $9}' access.log | sort | uniq -c | sort -rn
36
37 # 统计IP访问次数
38 awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10
```

TIP

awk 是一门完整的编程语言，适合复杂的文本处理任务。掌握 awk 可以大幅提升数据处理效率。

2.2 sed 高级用法

```
1  #!/bin/bash
2
3  # 多重编辑
4
5  sed -e 's/foo/bar/g' -e 's/baz/qux/g' file.txt
6
7  # 范围选择
8
9  sed '2,5d' file.txt          # 删除2-5行
10 sed '/start/,/end/d' file.txt # 删除start到end之间的行
11
12 # 插入和追加
13
14 sed '2i\New Line Before' file.txt # 在第2行前插入
15 sed '2a\New Line After' file.txt  # 在第2行后追加
16 sed '2c\Replace Line 2' file.txt  # 替换第2行
17
18 # 标签和分支（实现循环）
19
20 # 删除空行
21 sed '/^$/d' file.txt
22
23 # 合并连续空行为一行
24 sed '/^$/ {N; /\n$/d}' file.txt
25
26 # 反转文件行序
27
28 sed '1!G;h;$!d' file.txt
```

 Bash

```
29
30 # 实战: 批量修改配置文件
31
32 # 修改所有.conf文件中的端口
33 find /etc -name "*.conf" -exec sed -i 's/port=8080/port=9090/g' {} \;
34
35 # 注释掉特定行
36 sed -i '/^PermitRootLogin/s/^/#/' /etc/ssh/sshd_config
```

2.3 grep 高级用法

```
1  #!/bin/bash  Bash
2
3  # 扩展正则
4
5  grep -E "(error|warning|critical)" logfile.txt
6  grep -E "[0-9]{4}-[0-9]{2}-[0-9]{2}" logfile.txt
7
8  # Perl正则
9
10 grep -P "\d{4}-\d{2}-\d{2}" logfile.txt
11
12 # 上下文
13
14 grep -A 5 "error" logfile.txt # 显示匹配行及后5行
15 grep -B 5 "error" logfile.txt # 显示匹配行及前5行
16 grep -C 5 "error" logfile.txt # 显示匹配行及前后5行
17
18 # 递归搜索
19
20 grep -r "TODO" src/ # 递归搜索目录
21 grep -rl "TODO" src/ # 只显示文件名
22
23 # 排除文件
24
25 grep -r "pattern" --exclude="*.log" --exclude-dir="node_modules" .
26
27 # 计数和统计
28
29 grep -c "error" logfile.txt # 统计匹配行数
30 grep -o "error" logfile.txt | wc -l # 统计匹配次数
31
32 # 实战: 日志分析
33
34 # 查找最近1小时的错误
35 grep "$(date -d '1 hour ago' '+%Y-%m-%d %H%)" /var/log/syslog | grep error
36
37 # 统计各级别日志数量
38 grep -oP '\b(ERROR|WARN|INFO|DEBUG)\b' app.log | sort | uniq -c
```

3 定时任务

3.1 cron：周期性任务

```

1  # 编辑当前用户的 crontab
2
3  crontab -e
4
5  # 查看当前用户的 crontab
6
7  crontab -l
8
9  # 删除当前用户的 crontab
10
11 crontab -r
12
13 # 格式: 分 时 日 月 周 命令
14 # * * * * * command
15 # | | | | |
16 # | | | | +--- 星期 (0-7, 0和7都是周日)
17 # | | | +----- 月份 (1-12)
18 # | | +----- 日期 (1-31)
19 # | +----- 小时 (0-23)
20 # +----- 分钟 (0-59)
21
22 # 示例
23
24 # 每天凌晨2点执行
25 0 2 * * * /usr/local/bin/backup.sh
26
27 # 每15分钟执行
28 */15 * * * * /usr/local/bin/check_health.sh
29
30 # 每周一上午9点执行
31 0 9 * * 1 /usr/local/bin/weekly_report.sh
32
33 # 每月1号凌晨3点执行
34 0 3 1 * * /usr/local/bin/monthly_cleanup.sh
35
36 # 工作日每小时执行
37 0 * * * 1-5 /usr/local/bin/hourly_task.sh
38
39 # 特殊字符串
40 @reboot /usr/local/bin/startup.sh # 开机时执行
41 @yearly 0 0 1 1 * /usr/local/bin/yearly.sh # 每年执行
42 @monthly 0 0 1 * * /usr/local/bin/monthly.sh # 每月执行
43 @weekly 0 0 * * 0 /usr/local/bin/weekly.sh # 每周执行
44 @daily 0 0 * * * /usr/local/bin/daily.sh # 每天执行
45 @hourly 0 * * * * /usr/local/bin/hourly.sh # 每小时执行

```

CAUTION

cron 的环境变量很少，建议在脚本中使用绝对路径，或在 crontab 开头设置 PATH。

3.2 at: 一次性任务

```
1 # 在指定时间执行
2
3 at 14:30
4 at> /usr/local/bin/task.sh
5 at> <EOT> # Ctrl+D 结束
6
7 # 延迟执行
8
9 at now + 1 hour
10 at> command
11 at> <EOT>
12
13 at now + 3 days
14 at> command
15 at> <EOT>
16
17 # 查看待执行任务
18
19 atq
20
21 # 删除任务
22
23 atrm 123 # 删除任务ID为123的任务
24
25 # 从文件读取任务
26
27 echo "/usr/local/bin/task.sh" | at 15:00
```

3.3 systemd timers: 现代定时任务

```
1 # 创建 timer 单元文件
2
3 sudo tee /etc/systemd/system/backup.timer << EOF
4 [Unit]
5 Description=Daily Backup Timer
6
7 [Timer]
8 OnCalendar=*-*-* 02:00:00
9 Persistent=true
10
11 [Install]
12 WantedBy=timers.target
13 EOF
14
15 # 创建对应的 service 单元文件
16
```

```
17 sudo tee /etc/systemd/system/backup.service << EOF
18 [Unit]
19 Description=Daily Backup
20
21 [Service]
22 Type=oneshot
23 ExecStart=/usr/local/bin/backup.sh
24 EOF
25
26 # 启用并启动 timer
27
28 sudo systemctl enable backup.timer
29 sudo systemctl start backup.timer
30
31 # 查看 timer 状态
32
33 systemctl list-timers
34 systemctl list-timers --all
35
36 # 查看下次执行时间
37
38 systemctl status backup.timer
```

TIP

systemd timers 比 cron 更强大，支持更精确的时间控制、持久化、依赖管理等。

4 系统备份

4.1 tar: 归档压缩

```
1 #!/bin/bash
2
3 # 创建备份
4
5 tar czf backup-$(date +%Y%m%d).tar.gz /home/user/data
6
7 # 增量备份（需要配合 find）
8
9 find /home/user/data -mtime -1 -type f | tar czf incremental.tar.gz -T -
10
11 # 排除特定目录
12
13 tar czf backup.tar.gz --exclude='/tmp' --exclude='*.log' /home/user
14
15 # 查看归档内容
16
17 tar tzf backup.tar.gz
18
19 # 解压
```

 Bash

```
20
21 tar xzf backup.tar.gz
22 tar xzf backup.tar.gz -C /target/dir # 指定解压目录
23
24 # 保留权限
25
26 tar czpf backup.tar.gz /etc # p 保留权限
27
28 # 分卷压缩 (适合大文件)
29
30 tar czf - /large/dir | split -b 1G - backup.tar.gz.part.
31
32 # 恢复分卷
33
34 cat backup.tar.gz.part.* | tar xzf -
```

4.2 rsync: 增量同步

```
1 #!/bin/bash
2
3 # 本地同步
4
5 rsync -avz /source/ /destination/
6
7 # 远程同步 (SSH)
8
9 rsync -avz -e ssh /source/ user@remote:/destination/
10 rsync -avz -e ssh user@remote:/source/ /destination/
11
12 # 删除目标多余文件
13
14 rsync -avz --delete /source/ /destination/
15
16 # 排除文件
17
18 rsync -avz --exclude='*.log' --exclude='tmp/' /source/ /destination/
19
20 # 带宽限制
21
22 rsync -avz --bwlimit=1000 /source/ /destination/ # 1MB/s
23
24 # 断点续传
25
26 rsync -avz --partial --progress /source/ /destination/
27
28 # 备份脚本示例
29
30 #!/bin/bash
31
32 BACKUP_DIR="/backup"
33 SOURCE_DIR="/home/user/data"
```

 Bash

```
34 DATE=$(date +%Y%m%d_%H%M%S)
35
36 # 创建备份目录
37 mkdir -p $BACKUP_DIR
38
39 # 执行备份
40 rsync -avz --delete \
41     --exclude='*.tmp' \
42     --exclude='cache/' \
43     $SOURCE_DIR/ \
44     $BACKUP_DIR/backup-$DATE/
45
46 # 删除7天前的备份
47 find $BACKUP_DIR -maxdepth 1 -name "backup-*" -mtime +7 -exec rm -rf {} \;
48
49 echo "Backup completed: backup-$DATE"
```

TIP

rsync 只传输变化的部分，非常适合增量备份和大文件同步。

4.3 备份策略

```
1  #!/bin/bash
2
3  # 完整备份 + 增量备份策略
4
5  BACKUP_BASE="/backup"
6  SOURCE="/data"
7  DATE=$(date +%Y%m%d)
8  WEEKDAY=$(date +%u) # 1-7 (Monday-Sunday)
9
10 # 每周日完整备份
11 if [ $WEEKDAY -eq 7 ]; then
12     echo "Performing full backup..."
13     tar czf $BACKUP_BASE/full-$DATE.tar.gz $SOURCE
14 else
15     echo "Performing incremental backup..."
16     # 基于昨天的备份进行增量
17     YESTERDAY=$(date -d "yesterday" +%Y%m%d)
18     if [ -f "$BACKUP_BASE/full-$YESTERDAY.tar.gz" ]; then
19         find $SOURCE -newer $BACKUP_BASE/full-$YESTERDAY.tar.gz | \
20             tar czf $BACKUP_BASE/incr-$DATE.tar.gz -T -
21     fi
22 fi
23
24 # 清理旧备份（保留30天）
25 find $BACKUP_BASE -name "*.tar.gz" -mtime +30 -delete
26
27 echo "Backup completed on $(date)"
```

 Bash

备份策略建议：

策略	频率	优点	缺点
完全备份	每周	恢复简单	占用空间大
增量备份	每天	节省空间	恢复复杂
差异备份	每天	折中方案	后期备份变大
快照备份	实时	快速一致	需要文件系统支持

5 自动化部署脚本

5.1 SSH 免密登录

```
1  #!/bin/bash
2
3  # 生成密钥对（如果不存在）
4
5  if [ ! -f ~/.ssh/id_rsa ]; then
6      ssh-keygen -t rsa -b 4096 -N "" -f ~/.ssh/id_rsa
7  fi
8
9  # 复制公钥到远程服务器
10
11 ssh-copy-id user@remote-server
12
13 # 手动配置
14
15 cat ~/.ssh/id_rsa.pub | ssh user@remote-server "mkdir -p ~/.ssh && cat >> ~/.ssh/
authorized_keys"
16
17 # 测试免密登录
18
19 ssh user@remote-server "echo 'Success!'"
```

TIP

使用 `ssh-agent` 管理密钥，避免每次输入 passphrase。

5.2 批量执行命令

```
1  #!/bin/bash
2
3  # 服务器列表
4
5  SERVERS=(
6      "user@server1.example.com"
7      "user@server2.example.com"
8      "user@server3.example.com"
9  )
10
11 # 批量执行命令
```

```
12
13 for server in "${SERVERS[@]}"; do
14     echo "Executing on $server..."
15     ssh $server "uname -a && df -h && free -m"
16     echo "---"
17 done
18
19 # 并行执行 (使用 background jobs)
20
21 for server in "${SERVERS[@]}"; do
22     (
23         echo "Executing on $server..."
24         ssh $server "command"
25     ) &
26 done
27
28 wait # 等待所有后台任务完成
29
30 # 使用 parallel 工具 (更高效)
31
32 parallel ssh {} "command" ::: "${SERVERS[@]}"
```

5.3 自动化部署脚本

```
1  #!/bin/bash
2
3  set -euo pipefail # 严格模式
4
5  # 配置
6
7  APP_NAME="myapp"
8  APP_DIR="/opt/$APP_NAME"
9  REPO_URL="git@github.com:user/myapp.git"
10 BRANCH="main"
11 BACKUP_DIR="/backup/$APP_NAME"
12 DATE=$(date +%Y%m%d_%H%M%S)
13
14 # 颜色输出
15
16 RED='\033[0;31m'
17 GREEN='\033[0;32m'
18 YELLOW='\033[1;33m'
19 NC='\033[0m' # No Color
20
21 log_info() {
22     echo -e "${GREEN}[INFO]${NC} $1"
23 }
24
25 log_warn() {
26     echo -e "${YELLOW}[WARN]${NC} $1"
27 }
28
```

 Bash

```
29 log_error() {
30     echo -e "${RED}[ERROR]${NC} $1"
31 }
32
33 # 备份当前版本
34
35 backup_current() {
36     log_info "Backing up current version..."
37     mkdir -p $BACKUP_DIR
38     if [ -d "$APP_DIR" ]; then
39         tar czf $BACKUP_DIR/backup-$DATE.tar.gz -C $(dirname $APP_DIR) $(basename $APP_DIR)
40         log_info "Backup created: backup-$DATE.tar.gz"
41     fi
42 }
43
44 # 拉取最新代码
45
46 deploy_code() {
47     log_info "Deploying code..."
48
49     if [ ! -d "$APP_DIR" ]; then
50         git clone $REPO_URL $APP_DIR
51     else
52         cd $APP_DIR
53         git fetch origin
54         git reset --hard origin/$BRANCH
55     fi
56
57     log_info "Code deployed from branch: $BRANCH"
58 }
59
60 # 安装依赖
61
62 install_dependencies() {
63     log_info "Installing dependencies..."
64     cd $APP_DIR
65
66     # 根据项目类型选择
67     if [ -f "requirements.txt" ]; then
68         pip install -r requirements.txt
69     elif [ -f "package.json" ]; then
70         npm install --production
71     elif [ -f "pom.xml" ]; then
72         mvn clean package -DskipTests
73     fi
74
75     log_info "Dependencies installed"
76 }
77
78 # 重启服务
79
80 restart_service() {
81     log_info "Restarting service..."
```

```
82     sudo systemctl restart $APP_NAME
83     sudo systemctl status $APP_NAME --no-pager
84     log_info "Service restarted"
85 }
86
87 # 健康检查
88
89 health_check() {
90     log_info "Performing health check..."
91     sleep 5
92
93     if curl -f http://localhost:8080/health > /dev/null 2>&1; then
94         log_info "Health check passed"
95         return 0
96     else
97         log_error "Health check failed"
98         return 1
99     fi
100 }
101
102 # 回滚函数
103
104 rollback() {
105     log_warn "Rolling back to previous version..."
106
107     LATEST_BACKUP=$(ls -t $BACKUP_DIR/backup-*.tar.gz | head -1)
108     if [ -n "$LATEST_BACKUP" ]; then
109         tar xzf $LATEST_BACKUP -C /
110         sudo systemctl restart $APP_NAME
111         log_info "Rollback completed"
112     else
113         log_error "No backup found for rollback"
114         exit 1
115     fi
116 }
117
118 # 主流程
119
120 main() {
121     log_info "Starting deployment of $APP_NAME"
122
123     backup_current
124     deploy_code
125     install_dependencies
126     restart_service
127
128     if health_check; then
129         log_info "Deployment successful!"
130     else
131         log_error "Deployment failed, rolling back..."
132         rollback
133         exit 1
134     fi
135 }
```

```
135 }
136
137 # 执行
138
139 main "$@"
```

CAUTION

生产环境部署建议使用专业的 CI/CD 工具（Jenkins、GitLab CI、GitHub Actions），而非手写脚本。

5.4 监控告警脚本

```
1  #!/bin/bash
2
3  # 磁盘空间监控
4
5  check_disk() {
6      THRESHOLD=90
7
8      while read -r line; do
9          usage=$(echo $line | awk '{print $5}' | sed 's/%//')
10         mount=$(echo $line | awk '{print $6}')
11
12         if [ $usage -gt $THRESHOLD ]; then
13             echo "WARNING: Disk usage ${usage}% on $mount" | \
14                 mail -s "Disk Alert" admin@example.com
15         fi
16     done < <(df -h | tail -n +2)
17 }
18
19 # 内存监控
20
21 check_memory() {
22     THRESHOLD=90
23
24     used=$(free | awk '/Mem:/ {printf "%.0f", $3/$2 * 100}')
25
26     if [ $used -gt $THRESHOLD ]; then
27         echo "WARNING: Memory usage ${used}%" | \
28             mail -s "Memory Alert" admin@example.com
29     fi
30 }
31
32 # 服务监控
33
34 check_service() {
35     SERVICE=$1
36
37     if ! systemctl is-active --quiet $SERVICE; then
38         echo "CRITICAL: Service $SERVICE is not running" | \
39             mail -s "Service Alert: $SERVICE" admin@example.com
40 }
```

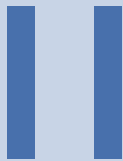
 Bash

```
41      # 尝试重启
42      systemctl restart $SERVICE
43  fi
44  }
45
46  # 执行检查
47
48  check_disk
49  check_memory
50  check_service nginx
51  check_service mysql
52
53  echo "Monitoring checks completed at $(date)"
```

本章完

Chapter 3

服务管理与高可用



容器化技术

4.	Docker 基础	39
5.	Docker 高级特性	53
6.	容器运行时	54

Chapter 4

Docker 基础

Docker 是最流行的容器化平台，通过轻量级容器实现应用的快速部署和隔离运行。

1 容器概念

1.1 容器 vs 虚拟机



特性	虚拟机	容器
启动速度	分钟级	秒级
资源占用	高 (完整 OS)	低 (共享内核)
隔离性	强	中 (命名空间)
性能损耗	5-10%	< 1%
镜像大小	GB 级	MB 级

密度	低	高
----	---	---

TIP

容器不是虚拟机的替代品，两者各有适用场景。容器适合微服务、CI/CD；虚拟机适合多租户、强隔离。

1.2 命名空间 (Namespaces)

命名空间提供资源隔离：

```

1 # 查看进程的命名空间
2 ls -l /proc/$$/ns
3
4 # 输出：
5 # lrwxrwxrwx 1 root root 0 Jan 1 12:00 cgroup -> cgroup:[4026531835]
6 # lrwxrwxrwx 1 root root 0 Jan 1 12:00 ipc -> ipc:[4026531839]
7 # lrwxrwxrwx 1 root root 0 Jan 1 12:00 mnt -> mnt:[4026531840]
8 # lrwxrwxrwx 1 root root 0 Jan 1 12:00 net -> net:[4026531992]
9 # lrwxrwxrwx 1 root root 0 Jan 1 12:00 pid -> pid:[4026531836]
10 # lrwxrwxrwx 1 root root 0 Jan 1 12:00 user -> user:[4026531837]
11 # lrwxrwxrwx 1 root root 0 Jan 1 12:00 uts -> uts:[4026531838]
```

6 种命名空间：

命名空间	隔离内容	作用
PID	进程 ID	进程可见性
NET	网络设备	网络栈隔离
MNT	挂载点	文件系统隔离
UTS	主机名	主机名隔离
IPC	进程间通信	信号量、消息队列
USER	用户 ID	用户权限映射

1.3 Cgroups (控制组)

Cgroups 限制和监控资源使用：

```

1 # 查看容器的 cgroup
2
3 docker inspect --format '{{.HostConfig.CpuShares}}' container_id
4 docker inspect --format '{{.HostConfig.Memory}}' container_id
5
6 # 限制 CPU 和内存
7
8 docker run --cpus=1.5 --memory=512m nginx
9
10 # 查看 cgroup 文件系统
11
12 cat /sys/fs/cgroup/cpu/docker/<container_id>/cpu.shares
13 cat /sys/fs/cgroup/memory/docker/<container_id>/memory.limit_in_bytes
```

Cgroups v1 vs v2:

- v1: 多个子系统, 层次结构复杂
- v2: 统一 hierarchy, 更简洁
- Docker 支持两种版本

NOTE

命名空间提供隔离, Cgroups 提供资源限制, 两者结合实现容器化。

2 Docker 安装与配置

2.1 安装 Docker Engine

```
1 # Ubuntu/Debian
2
3 # 卸载旧版本
4 sudo apt-get remove docker docker-engine docker.io containerd runc
5
6 # 安装依赖
7 sudo apt-get update
8 sudo apt-get install \
9     ca-certificates \
10    curl \
11    gnupg \
12    lsb-release
13
14 # 添加 GPG 密钥
15 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/
16    keyrings/docker-archive-keyring.gpg
17
18 # 添加仓库
19 echo \
20     "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-
21     keyring.gpg] https://download.docker.com/linux/ubuntu \
22     $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
23
24 # 安装
25 sudo apt-get update
26 sudo apt-get install docker-ce docker-ce-cli containerd.io
27
28 # 验证
29 sudo docker run hello-world
```

CAUTION

生产环境建议使用官方脚本或包管理器安装, 避免使用 convenience scripts。

2.2 Docker 守护进程配置

```
1 # 配置文件位置
```

```

2
3  sudo vim /etc/docker/daemon.json
4
5  # 示例配置
6  {
7      "registry-mirrors": [
8          "https://docker.mirrors.ustc.edu.cn",
9          "https://hub-mirror.c.163.com"
10     ],
11     "storage-driver": "overlay2",
12     "log-driver": "json-file",
13     "log-opts": {
14         "max-size": "10m",
15         "max-file": "3"
16     },
17     "default-runtime": "runc",
18     "live-restore": true
19 }
20
21 # 重启 Docker
22
23 sudo systemctl daemon-reload
24 sudo systemctl restart docker
25
26 # 验证配置
27
28 docker info

```

常用配置项：

配置项	说明	推荐值
registry-mirrors	镜像加速	国内镜像源
storage-driver	存储驱动	overlay2
log-driver	日志驱动	json-file
max-size	日志大小	10m
live-restore	守护进程重启时保持容器运行	true

2.3 镜像加速器配置

```

1  # 国内常用镜像源
2
3  # 阿里云（需要注册获取专属地址）
4  https://<your-id>.mirror.aliyuncs.com
5
6  # 中科大
7  https://docker.mirrors.ustc.edu.cn
8
9  # 网易
10 https://hub-mirror.c.163.com

```

```
11
12 # 腾讯云
13 https://mirror.ccs.tencentyun.com
14
15 # 配置后测试
16
17 docker pull nginx # 应该明显加快
```

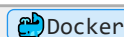
TIP

配置镜像加速器可以大幅提升镜像拉取速度，特别是国内环境。

3 镜像管理

3.1 Dockerfile 基础

```
1 # 基础镜像
2 FROM ubuntu:22.04
3
4 # 元数据
5 LABEL maintainer="alice@example.com"
6 LABEL version="1.0"
7 LABEL description="My application"
8
9 # 环境变量
10 ENV APP_HOME=/app
11 ENV NODE_ENV=production
12
13 # 工作目录
14 WORKDIR $APP_HOME
15
16 # 复制文件
17 COPY package*.json ./
18 COPY . .
19
20 # 执行命令
21 RUN apt-get update && \
22     apt-get install -y --no-install-recommends \
23     nodejs npm && \
24     rm -rf /var/lib/apt/lists/* && \
25     npm install
26
27 # 暴露端口
28 EXPOSE 3000
29
30 # 卷
31 VOLUME ["/data"]
32
33 # 健康检查
34 HEALTHCHECK --interval=30s --timeout=3s \
```



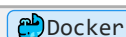
```
35 CMD curl -f http://localhost:3000/ || exit 1
36
37 # 启动命令
38 CMD ["node", "server.js"]
```

Dockerfile 最佳实践:

1. 使用具体的基础镜像标签 (不用 `latest`)
2. 合并 RUN 指令, 减少层数
3. 使用 `.dockerignore` 排除不必要文件
4. 多阶段构建减小镜像体积
5. 不以 root 用户运行

3.2 分层存储原理

```
1 # 每一层都是只读的
2 FROM ubuntu:22.04 # Layer 1: 基础镜像
3 RUN apt-get update # Layer 2: 更新包列表
4 RUN apt-get install -y nginx # Layer 3: 安装 nginx
5 COPY . /var/www/html # Layer 4: 复制文件
6 EXPOSE 80 # Layer 5: 元数据
7 CMD ["nginx", "-g", "daemon off;"] # Layer 6: 启动命令
```



分层优势:

- 缓存: 未改变的层可以复用
- 共享: 多个镜像可以共享相同的层
- 增量更新: 只传输变化的层

```
1 # 查看镜像层
2
3 docker history nginx:latest
4
5 # 输出:
6 # IMAGE          CREATED          CREATED BY                                      SIZE
7 # abc123         2 days ago     /bin/sh -c #(nop) CMD ["nginx" "-g" ...]    0B
8 # def456         2 days ago     /bin/sh -c #(nop) EXPOSE 80                  0B
9 # ghi789         2 days ago     /bin/sh -c apt-get update && apt-get ...    50MB
10 # jkl012         1 week ago     /bin/sh -c #(nop) FROM ubuntu:22.04         77MB
```

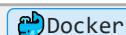


NOTE

理解分层存储有助于优化 Dockerfile, 减小镜像体积, 加快构建速度。

3.3 镜像优化

```
1 # ❌ 不好的实践: 多层、大体积
2 FROM ubuntu:22.04
3 RUN apt-get update
4 RUN apt-get install -y python3
5 RUN apt-get install -y python3-pip
6 RUN pip3 install flask
7 COPY . /app
```



```

8
9 # ✅ 好的实践：合并层、清理缓存
10 FROM ubuntu:22.04
11 RUN apt-get update && \
12     apt-get install -y --no-install-recommends \
13     python3 python3-pip && \
14     rm -rf /var/lib/apt/lists/* && \
15     pip3 install --no-cache-dir flask
16 COPY . /app
17
18 # ✅ 更好的实践：多阶段构建
19 # 构建阶段
20 FROM node:18-alpine AS builder
21 WORKDIR /app
22 COPY package*.json ./
23 RUN npm ci --production
24 COPY . .
25 RUN npm run build
26
27 # 运行阶段
28 FROM nginx:alpine
29 COPY --from=builder /app/dist /usr/share/nginx/html
30 EXPOSE 80
31 CMD ["nginx", "-g", "daemon off;"]

```

优化技巧：

技巧	效果	示例
使用 Alpine	减小体积	python:3.11-alpine
多阶段构建	去除构建工具	builder + runtime
合并 RUN	减少层数	apt-get update && install
清理缓存	减小体积	rm -rf /var/lib/apt/lists/*
.dockerignore	减小上下文	排除 node_modules, .git
不使用 root	安全性	USER nobody

3.4 镜像操作命令

```

1 # 拉取镜像
2
3 docker pull nginx:latest
4 docker pull python:3.11-slim
5
6 # 查看本地镜像
7
8 docker images
9 docker images -a # 包括中间层
10
11 # 搜索镜像

```

 Bash

```
12
13 docker search nginx
14
15 # 删除镜像
16
17 docker rmi nginx:latest
18 docker rmi <image_id>
19
20 # 导出/导入
21
22 docker save nginx > nginx.tar
23 docker load < nginx.tar
24
25 # 打标签
26
27 docker tag nginx:latest myregistry.com/nginx:v1.0
28
29 # 推送到仓库
30
31 docker push myregistry.com/nginx:v1.0
```

4 容器操作

4.1 运行容器

```
1 # 基本运行 🖥️ Bash
2
3 docker run nginx
4
5 # 后台运行
6
7 docker run -d nginx
8
9 # 指定名称
10
11 docker run -d --name my-nginx nginx
12
13 # 端口映射
14
15 docker run -d -p 8080:80 nginx # 主机8080 -> 容器80
16
17 # 环境变量
18
19 docker run -d -e MYSQL_ROOT_PASSWORD=secret mysql
20
21 # 挂载卷
22
23 docker run -d -v /host/data:/container/data nginx
24
25 # 资源限制
```

```
26
27 docker run -d --cpus=1.5 --memory=512m nginx
28
29 # 网络
30
31 docker run -d --network my-network nginx
32
33 # 重启策略
34
35 docker run -d --restart=always nginx
```

TIP

常用组合: `docker run -d --name <name> -p <host_port>:<container_port> <image>`

4.2 容器生命周期管理

```
1 # 查看运行中的容器
2
3 docker ps
4 docker ps -a # 包括停止的
5
6 # 停止容器
7
8 docker stop my-nginx
9 docker stop $(docker ps -q) # 停止所有
10
11 # 启动已停止的容器
12
13 docker start my-nginx
14
15 # 重启容器
16
17 docker restart my-nginx
18
19 # 删除容器
20
21 docker rm my-nginx
22 docker rm $(docker ps -aq) # 删除所有
23
24 # 强制删除运行中的容器
25
26 docker rm -f my-nginx
```

 Bash

4.3 exec 进入容器

```
1 # 执行命令
2
3 docker exec my-nginx ls /usr/share/nginx/html
4
5 # 交互式 shell
```

 Bash

```
6
7  docker exec -it my-nginx bash
8  docker exec -it my-nginx sh  # Alpine 镜像没有 bash
9
10 # 以特定用户执行
11
12 docker exec -u root -it my-nginx bash
13
14 # 环境变量
15
16 docker exec -e MY_VAR=value my-nginx env
```

CAUTION

生产环境尽量避免 exec 进入容器调试，应该通过日志和监控排查问题。

4.4 日志查看

```
1  # 查看日志
2
3  docker logs my-nginx
4
5  # 实时跟踪
6
7  docker logs -f my-nginx
8
9  # 显示时间戳
10
11 docker logs -t my-nginx
12
13 # 最后100行
14
15 docker logs --tail 100 my-nginx
16
17 # 最近1小时
18
19 docker logs --since 1h my-nginx
20
21 # 导出日志
22
23 docker logs my-nginx > nginx.log 2>&1
```

NOTE

默认日志驱动是 json-file，可以通过 daemon.json 配置日志轮转。

4.5 inspect 查看详情

```
1  # 查看容器详细信息
2
3  docker inspect my-nginx
4
```

```
5 # 格式化输出
6
7 docker inspect --format='{{.NetworkSettings.IPAddress}}' my-nginx
8 docker inspect --format='{{.State.Status}}' my-nginx
9 docker inspect --format='{{.Config.Env}}' my-nginx
10
11 # 查看镜像详情
12
13 docker inspect nginx:latest
14
15 # JSON 格式
16
17 docker inspect my-nginx | jq .
```

5 数据持久化

5.1 Volume（卷）

```
1 # 创建卷
2
3 docker volume create my-data
4
5 # 查看卷
6
7 docker volume ls
8 docker volume inspect my-data
9
10 # 使用卷
11
12 docker run -d -v my-data:/app/data nginx
13
14 # 删除卷
15
16 docker volume rm my-data
17 docker volume prune # 删除未使用的卷
```

 Bash

Volume 特点：

- 由 Docker 管理，存储在 `/var/lib/docker/volumes/`
- 易于备份和迁移
- 支持多种驱动（local、nfs、cloud 等）

5.2 Bind Mount（绑定挂载）

```
1 # 绑定主机目录
2
3 docker run -d -v /host/path:/container/path nginx
4
5 # 只读挂载
6
```

 Bash

```
7 docker run -d -v /host/path:/container/path:ro nginx
8
9 # 挂载单个文件
10
11 docker run -d -v /host/nginx.conf:/etc/nginx/nginx.conf nginx
```

Bind Mount 特点:

- 直接映射主机文件系统
- 性能好, 适合开发环境
- 路径依赖主机, 可移植性差

TIP

开发环境用 Bind Mount (代码热更新), 生产环境用 Volume (数据持久化)。

5.3 tmpfs (临时文件系统)

```
1 # 使用 tmpfs
2
3 docker run -d --tmpfs /app/tmp nginx
4
5 # 指定大小
6
7 docker run -d --tmpfs /app/tmp:size=100m nginx
```

tmpfs 特点:

- 存储在内存中
- 容器停止后数据丢失
- 适合敏感数据 (密码、token)

5.4 数据卷容器模式

```
1 # 创建数据卷容器
2
3 docker create -v /data --name data-store alpine /bin/true
4
5 # 其他容器挂载
6
7 docker run -d --volumes-from data-store --name app1 nginx
8 docker run -d --volumes-from data-store --name app2 nginx
9
10 # 备份
11
12 docker run --rm --volumes-from data-store -v $(pwd):/backup alpine \
13 tar czf /backup/backup.tar.gz /data
```

NOTE

数据卷容器模式在 Docker Compose 出现前常用, 现在推荐使用命名卷。

6 网络模式

6.1 Bridge 模式（默认）

```
1 # 创建自定义网桥
2
3 docker network create my-bridge
4
5 # 连接到网桥
6
7 docker run -d --network my-bridge --name web nginx
8 docker run -d --network my-bridge --name db mysql
9
10 # 容器间通信（通过容器名）
11
12 docker exec web ping db # 可以解析容器名
13
14 # 查看网络
15
16 docker network ls
17 docker network inspect my-bridge
```

Bridge 特点：

- 默认网络模式
- 容器有独立 IP
- 需要通过端口映射访问
- 自定义网桥支持 DNS 解析

6.2 Host 模式

```
1 # 使用 host 网络
2
3 docker run -d --network host nginx
4
5 # 容器直接使用主机网络栈
6 # 无需端口映射
7 # 访问 localhost:80 即可
```

Host 特点：

- 性能最好（无 NAT）
- 端口冲突风险
- 隔离性差
- 适合高性能需求场景

6.3 None 模式

```
1 # 禁用网络
2
3 docker run -d --network none alpine sleep 3600
```

```
4
5 # 容器只有 loopback 接口
6 # 适合批处理任务
```

6.4 Overlay 模式 (Swarm)

```
1 # 初始化 Swarm
2
3 docker swarm init
4
5 # 创建 overlay 网络
6
7 docker network create -d overlay my-overlay
8
9 # 跨主机通信
10
11 docker service create --network my-overlay --name web nginx
```

Overlay 特点:

- 跨主机容器通信
- 用于 Docker Swarm
- Kubernetes 使用 CNI 插件

模式	隔离性	性能	适用场景
bridge	中	中	默认, 通用
host	低	高	高性能需求
none	高	N/A	无网络需求
overlay	中	中	跨主机通信

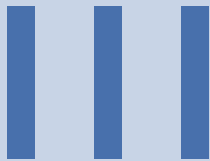
本章完

Chapter 5

Docker 高级特性

Chapter 6

容器运行时



云原生基础

7. 云原生理念与架构	56
8. 服务网格 (Service Mesh)	57
9. 云原生存储	58

Chapter 7

云原生理念与架构

Chapter 8

服务网格（Service Mesh）

Chapter 9

云原生存储

IV

Kubernetes 核心

10. K8s 架构与核心概念	60
11. 工作负载管理	61
12. 服务发现与网络	62
13. 存储与配置管理	63
14. 安全与权限控制	64

Chapter 10

K8s 架构与核心概念

Chapter 11

工作负载管理

Chapter 12

服务发现与网络

Chapter 13

存储与配置管理

Chapter 14

安全与权限控制



Kubernetes 进阶

15. Helm 包管理	66
16. Operator 模式	67
17. 监控与日志	68
18. CI/CD 与 GitOps	69

Chapter 15

Helm 包管理

Chapter 16

Operator 模式

Chapter 17

监控与日志

Chapter 18

CI/CD 与 GitOps



云平台与基础设施

19. 公有云 K8s 服务	71
20. 基础设施即代码 (IaC)	72
21. 边缘计算与 K8s	73

Chapter 19

公有云 **K8s** 服务

Chapter 20

基础设施即代码 (IaC)

Chapter 21

边缘计算与 K8s

VII

生产实践与故障排查

22. 集群规划与部署	75
23. 性能优化	76
24. 故障排查方法论	77
25. 成本优化与管理	78

Chapter 22

集群规划与部署

Chapter 23

性能优化

Chapter 24

故障排查方法论

Chapter 25

成本优化与管理
